



Vidyavardhini's College of Engineering and Technology, Vasai

Department of Computer Science & Engineering (Data Science)

EXPERIMENT ASSESSMENT

ACADEMIC YEAR 2025-26

Course: Introduction to Web Technology

Course code: 2344211

Year: SE SEM: IV

Experiment No. 9
Aim: Implement search functionality to filter API results dynamically.
Name: Hetvi Patel
Roll Number: 56
Date of Performance:
Date of Submission:

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission.	10	
Total	20	

Checked by

Name of Faculty :

Signature :

Date



Experiment 9

Title

Implement search functionality to filter API results dynamically.

Theory:

Modern web applications often handle large amounts of data retrieved from external APIs. Displaying all fetched data at once can reduce usability. To improve user experience, search and filtering functionality is implemented to dynamically narrow down results based on user input.

An API (Application Programming Interface) provides structured data in formats such as JSON. JavaScript fetches this data asynchronously and stores it in memory. A search input field allows users to enter keywords, which are used to filter the API results in real time.

1. Dynamic Search Filtering

Dynamic search filtering allows data to be filtered without refreshing the page. As the user types in the search box, JavaScript compares the input text with the fetched data and displays only matching records.

2. AJAX and Fetch API

The Fetch API is used to retrieve data asynchronously from the server. It returns a promise that resolves with the API response, which is then converted into JSON format. This ensures smooth and fast data retrieval.

3. JavaScript Array Methods

JavaScript provides built-in array methods such as `filter()` and `includes()` to efficiently search through data. These methods help compare user input with data fields like name, title, or description.

4. DOM Manipulation

The filtered data is displayed dynamically using DOM manipulation, ensuring instant updates to the user interface whenever the search input changes.

Algorithm / Procedure



Vidyavardhini's College of Engineering and Technology, Vasai

Department of Computer Science & Engineering (Data Science)

1. Start the experiment.
2. Create an HTML file with a search input field.
3. Use JavaScript to fetch data from a public API.
4. Store the API response in an array.
5. Display all API results on the webpage.
6. Capture user input from the search field.
7. Filter the stored data based on the search keyword.
8. Update the displayed results dynamically.
9. Repeat filtering as the user types.
10. Stop the experiment.

Program / Code:

```
<!DOCTYPE html>

<html>

<head>

  <title>API Search Filter</title>

  <style>

    body {

      font-family: Arial;

      background: #f4f4f4;

    }

    input {

      width: 300px;

      padding: 10px;
```



Vidyavardhini's College of Engineering and Technology, Vasai

Department of Computer Science & Engineering (Data Science)

```
margin: 20px;
}
.card {
  background: white;
  padding: 15px;
  margin: 10px;
  border-radius: 5px;
}
</style>
</head>
<body>

<h2 align="center">Search API Data</h2>
<center>
<input type="text" id="search" placeholder="Search by title...">
</center>

<div id="results"></div>

<script>
let apiData = [];

fetch("https://jsonplaceholder.typicode.com/posts")
.then(response => response.json())
.then(data => {
```



Vidyavardhini's College of Engineering and Technology, Vasai

Department of Computer Science & Engineering (Data Science)

```
    apiData = data;
    displayData(apiData);
});

function displayData(data) {
    const resultDiv = document.getElementById("results");
    resultDiv.innerHTML = "";

    data.forEach(item => {
        resultDiv.innerHTML += `
            <div class="card">
                <h4>${item.title}</h4>
                <p>${item.body}</p>
            </div>
        `;
    });
}

document.getElementById("search").addEventListener("keyup", function() {
    const keyword = this.value.toLowerCase();
    const filteredData = apiData.filter(item =>
        item.title.toLowerCase().includes(keyword)
    );
    displayData(filteredData);
});
```



Vidyavardhini's College of Engineering and Technology, Vasai

Department of Computer Science & Engineering (Data Science)

</script>

</body>

</html>

Output:

Search API Data

sunt aut facere repellat provident occaecati excepturi optio reprehenderit

quia et suscipit suscipit recusandae consequuntur expedita et cum reprehenderit molestiae ut ut quas totam nostrum rerum est autem sunt rem eveniet architecto

qui est esse

est rerum tempore vitae sequi sint nihil reprehenderit dolor beatae ea dolores neque fugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis qui aperiam non debitis possimus qui neque nisi nulla

ea molestias quasi exercitationem repellat qui ipsa sit aut

et iusto sed quo iure voluptatem occaecati omnis eligendi aut ad voluptatem doloribus vel accusantium quis pariatur molestiae porro eius odio et labore et velit aut

eum et est occaecati

ullam et saepe reiciendis voluptatem adipisci sit amet autem assumenda provident rerum culpa quia hic commodi nesciunt rem tenetur doloremque ipsam iure quae sunt voluptatem rerum ito velit

Conclusion

Thus, dynamic search functionality was successfully implemented to filter API results using JavaScript. This experiment demonstrates the use of Fetch API, array filtering methods, and DOM manipulation to create responsive and interactive web applications.



Vidyavardhini's College of Engineering and Technology, Vasai

Department of Computer Science & Engineering (Data Science)

Question:

1. What is Dynamic Search Filtering?

Dynamic search filtering means showing results instantly as the user types in a search box. The list updates in real-time without reloading the page.

2. How is DOM manipulation used in this experiment?

DOM manipulation is used to change the content of the webpage (like showing/hiding items) based on user input.

3. Why is page reload not required?

Page reload is not required because JavaScript updates the content directly in the browser using DOM manipulation.